GURUGRAM UNIVERSITY — DEPARTMENT OF ENGINEERING & TECHNOLOGY

Neural Networks

Concepts, Architectures and Applications

Dr. Mukesh Kumar

Associate Professor

Department of Engineering & Technology

Gurugram University, Gurugram, Haryana

Structured in Four Modules for B.Tech / M.Tech Students

© 2026 Dr. Mukesh Kumar. All rights reserved.

Preface

This book provides a comprehensive, module-based introduction to neural networks for undergraduate and postgraduate engineering students. Organised into four modules, it guides the reader from biological inspiration and mathematical foundations through to advanced architectures and real-world applications.

Module 1 covers the historical development, biological motivation, and fundamental structure of artificial neural networks. Module 2 focuses on the learning algorithms — perceptrons, multi-layer networks, and the celebrated backpropagation algorithm. Module 3 delves into activation functions, regularisation, and optimisation strategies essential for effective training. Module 4 presents Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and a survey of modern deep learning applications.

Each module is self-contained and includes definitions, theorems, worked examples, and review questions. Students are expected to have basic familiarity with calculus, linear algebra, and programming.

Dr. Mukesh Kumar
Associate Professor, Department of Engineering & Technology
Gurugram University

Contents

1 Module 1: Foundations of Neural Networks **1**
- 1.1 Historical Development 1
- 1.2 Biological Motivation 2
- 1.3 The Artificial Neuron 2
- 1.4 Network Architecture 3
- 1.5 Types of Neural Networks 3
- 1.6 Module 1 — Review Questions 4

2 Module 2: Learning Algorithms **5**
- 2.1 The Perceptron 5
  - 2.1.1 Perceptron Learning Rule 5
  - 2.1.2 Limitations of the Perceptron 5
- 2.2 Multi-Layer Perceptron 6
  - 2.2.1 Forward Propagation 6
- 2.3 Loss Functions 6
  - 2.3.1 Regression: Mean Squared Error 7
  - 2.3.2 Binary Classification: Binary Cross-Entropy 7
  - 2.3.3 Multi-class Classification: Categorical Cross-Entropy 7
- 2.4 Backpropagation 7
  - 2.4.1 Derivation 7
  - 2.4.2 Variants of Gradient Descent 8
- 2.5 Module 2 — Review Questions 8

3 Module 3: Training Deep Networks **9**
- 3.1 Activation Functions 9
  - 3.1.1 Sigmoid 9
  - 3.1.2 Hyperbolic Tangent (tanh) 9
  - 3.1.3 Rectified Linear Unit (ReLU) 9
  - 3.1.4 Softmax 10
- 3.2 Regularisation Techniques 10

3.2.1 L2 Regularisation (Weight Decay) 10
3.2.2 Dropout 10
3.2.3 Batch Normalisation 11
3.3 Optimisation Algorithms 11
3.3.1 Momentum 11
3.3.2 Adam 11
3.4 Hyperparameter Tuning 12
3.5 Module 3 — Review Questions 12

4 Module 4: Advanced Architectures and Applications 13
4.1 Convolutional Neural Networks (CNNs) 13
4.1.1 Convolutional Layer 13
4.1.2 Pooling Layer 13
4.1.3 Landmark CNN Architectures 13
4.2 Recurrent Neural Networks (RNNs) 14
4.2.1 Long Short-Term Memory (LSTM) 14
4.2.2 Gated Recurrent Unit (GRU) 15
4.3 Transformer Architecture 15
4.4 Applications of Neural Networks 15
4.5 Challenges and Future Directions 16
4.6 Module 4 — Review Questions 16

Module 1: Foundations of Neural Networks

1.1 Historical Development

The study of artificial neural networks (ANNs) has a rich history stretching over seven decades. In 1943, Warren McCulloch and Walter Pitts proposed a mathematical model of the biological neuron, demonstrating that networks of threshold logic units could compute any Boolean function. This landmark paper laid the conceptual groundwork for all subsequent work in the field.

Donald Hebb, in 1949, put forward the first biologically motivated learning rule — now known as Hebbian learning — which states that the synaptic connection between two neurons strengthens when they fire simultaneously. This "neurons that fire together, wire together" principle remains influential in modern unsupervised learning algorithms.

Frank Rosenblatt (1957) implemented the first trainable neural network, the *Perceptron*, on the IBM 704 computer. The perceptron was capable of learning to classify linearly separable patterns and attracted enormous optimism. However, Minsky and Papert's analysis (1969) exposed the perceptron's inability to learn the XOR function, triggering the first "AI winter" — a prolonged decline in neural network research funding.

The renaissance came with the rediscovery and formalisation of the backpropagation algorithm by Rumelhart, Hinton, and Williams in 1986. Multi-layer networks trained by backpropagation could, for the first time, represent and learn complex non-linear functions. The 1990s introduced Support Vector Machines as strong competitors, yet by 2012, deep neural networks achieved human-level performance on ImageNet, launching the current era of deep learning.

Since then, transformers, large language models, and generative adversarial networks have redefined the boundaries of artificial intelligence.

1.2 Biological Motivation

The human brain contains approximately 10^{11} neurons, interconnected through roughly 10^{14} synaptic connections. Each biological neuron consists of:

- **Dendrites:** Tree-like projections that receive electrochemical signals from other neurons.
- **Soma (Cell body):** Integrates incoming signals; if the total exceeds a threshold, the neuron fires.
- **Axon:** A long fibre that transmits the electrical impulse (action potential) to other neurons.
- **Synapses:** Junctions between axon terminals and dendrites of the receiving neuron; their efficacy determines connection strength.

Artificial neural networks model this structure. Each artificial neuron receives numerical inputs, computes a weighted sum, applies an activation function, and produces an output. Although heavily simplified, this model captures the distributed, parallel nature of biological computation and its remarkable capacity to learn from experience.

1.3 The Artificial Neuron

Definition 1.1 (Artificial Neuron)**.** An artificial neuron is a computational unit that computes a weighted sum of its inputs, adds a bias term, and applies an activation function to produce an output:

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right) = f(\mathbf{w}^T \mathbf{x} + b) \tag{1.1}$$

where $\mathbf{x} = (x_1, \ldots, x_n)^T$ is the input vector, $\mathbf{w} = (w_1, \ldots, w_n)^T$ is the weight vector, b is the bias, and $f(\cdot)$ is the activation function.

The weight w_i represents the importance of input x_i; a large positive weight amplifies the signal, a negative weight inhibits it, and a weight near zero effectively ignores it. The bias allows the activation function to be shifted, providing the model with an extra degree of freedom.

1.4 Network Architecture

Neurons in an ANN are organised into **layers**:

Input Layer Receives raw features from the external world. No computation occurs; it simply distributes inputs to the next layer.

Hidden Layer(s) Intermediate layers where complex transformations are learned. The term "hidden" reflects that their outputs are not directly observed.

Output Layer Produces the network's final prediction — a class label, a real value, or a probability distribution.

A network with one hidden layer is called a *shallow* network; networks with two or more hidden layers are called *deep* networks, giving rise to the term "deep learning."

1.5 Types of Neural Networks

1. **Feedforward Networks (FNN):** Data flows strictly from input to output. Includes single-layer perceptrons and multi-layer perceptrons (MLPs). Suitable for tabular data and fixed-size inputs.

2. **Convolutional Neural Networks (CNN):** Exploit spatial structure through convolutional filters and pooling. Pre-eminent for image and video data.

3. **Recurrent Neural Networks (RNN):** Incorporate feedback connections that give the network a form of memory. Suited to sequential data such as text, speech, and time series.

4. **Autoencoders:** Trained to reconstruct their input through a bottleneck, learning a compressed representation. Used for dimensionality reduction and anomaly detection.

5. **Generative Adversarial Networks (GAN):** Two networks — a generator and a discriminator — trained adversarially to produce realistic synthetic data.

6. **Transformer Networks:** Attention-based architecture that processes sequences in parallel. Foundation of modern large language models (BERT, GPT, LLaMA).

1.6 Module 1 — Review Questions

1. Describe the McCulloch–Pitts neuron model and its significance.

2. What is Hebbian learning? How does it differ from supervised learning?

3. Explain the contributions of Rosenblatt and their impact on neural network research.

4. Distinguish between shallow and deep neural networks with an example.

5. How does the artificial neuron model the biological neuron? What are its limitations?

Module 2: Learning Algorithms

2.1 The Perceptron

The perceptron, proposed by Frank Rosenblatt in 1957, is the foundational supervised learning algorithm for binary classification. Given input vector $\mathbf{x} \in \mathbb{R}^n$, it outputs:

$$\hat{y} = \begin{cases} 1 & \text{if } \mathbf{w}^T\mathbf{x} + b \geq 0 \\ 0 & \text{otherwise} \end{cases} \tag{2.1}$$

2.1.1 Perceptron Learning Rule

Given a labelled training example $(\mathbf{x}, y)$ where $y \in \{0, 1\}$, the weights are updated as:

$$w_i \leftarrow w_i + \eta\,(y - \hat{y})\,x_i, \qquad b \leftarrow b + \eta\,(y - \hat{y}) \tag{2.2}$$

where $\eta > 0$ is the learning rate. If the prediction is correct ($y = \hat{y}$), no update is made.

Theorem 2.1 (Perceptron Convergence Theorem)**.** *If the training data is linearly separable, the perceptron learning algorithm converges to a correct separating hyperplane in a finite number of weight updates.*

2.1.2 Limitations of the Perceptron

The perceptron can only learn linearly separable decision boundaries. A canonical failure case is the XOR function:

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

No linear classifier can correctly partition these four points. This limitation motivated the development of multi-layer networks.

2.2 Multi-Layer Perceptron

A multi-layer perceptron (MLP) augments the single-layer perceptron with one or more hidden layers of non-linear units, enabling it to model complex, non-linear decision boundaries.

2.2.1 Forward Propagation

Let layer l have weight matrix $\mathbf{W}^{(l)} \in \mathbb{R}^{m_l \times m_{l-1}}$ and bias $\mathbf{b}^{(l)}$. The pre-activation and activation of layer l are:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \tag{2.3}$$

$$\mathbf{a}^{(l)} = f^{(l)}(\mathbf{z}^{(l)}) \tag{2.4}$$

with $\mathbf{a}^{(0)} = \mathbf{x}$ (the input). The final output $\hat{\mathbf{y}} = \mathbf{a}^{(L)}$ where L is the number of layers.

Theorem 2.2 (Universal Approximation)**.** *A feedforward network with at least one hidden layer and a non-polynomial activation function can approximate any continuous function $f : \mathbb{R}^n \to \mathbb{R}^m$ on a compact subset of $\mathbb{R}^n$, to arbitrary precision, given sufficiently many hidden neurons.*

2.3 Loss Functions

The choice of loss function depends on the task:

2.3.1 Regression: Mean Squared Error

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{k=1}^{N}(y_k - \hat{y}_k)^2 \tag{2.5}$$

2.3.2 Binary Classification: Binary Cross-Entropy

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N}\sum_{k=1}^{N}\big[y_k \log \hat{y}_k + (1 - y_k)\log(1 - \hat{y}_k)\big] \tag{2.6}$$

2.3.3 Multi-class Classification: Categorical Cross-Entropy

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N}\sum_{k=1}^{N}\sum_{c=1}^{C} y_{kc}\ \log \hat{y}_{kc} \tag{2.7}$$

2.4 Backpropagation

Backpropagation is an efficient algorithm for computing the gradient $\nabla_{\mathbf{W}}\mathcal{L}$ by the chain rule, propagating errors from the output layer backward to each hidden layer.

2.4.1 Derivation

Define the error signal at layer l as $\boldsymbol{\delta}^{(l)} = \partial\mathcal{L}/\partial\mathbf{z}^{(l)}$.

Output layer:

$$\boldsymbol{\delta}^{(L)} = \nabla_{\mathbf{a}^{(L)}}\mathcal{L} \odot f^{(L)\prime}(\mathbf{z}^{(L)}) \tag{2.8}$$

Hidden layers ($l = L-1, L-2, \ldots, 1$):

$$\boldsymbol{\delta}^{(l)} = \big((\mathbf{W}^{(l+1)})^T\boldsymbol{\delta}^{(l+1)}\big) \odot f^{(l)\prime}(\mathbf{z}^{(l)}) \tag{2.9}$$

Gradient computation:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)}\,(\mathbf{a}^{(l-1)})^T, \qquad \frac{\partial\mathcal{L}}{\partial\mathbf{b}^{(l)}} = \boldsymbol{\delta}^{(l)} \tag{2.10}$$

Weight update (gradient descent):

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} \tag{2.11}$$

2.4.2 Variants of Gradient Descent

Batch Gradient Descent Uses the entire training set per update. Stable gradients but expensive for large datasets.

Stochastic Gradient Descent (SGD) Updates after every single sample. Fast and noisy; can escape local minima.

Mini-Batch SGD Uses a small subset (mini-batch) of 32–256 samples. Balances stability and speed; the standard approach in practice.

2.5 Module 2 — Review Questions

1. Derive the perceptron learning rule from first principles.
2. Explain why non-linear activation functions are essential in MLPs.
3. Describe the forward propagation algorithm for a three-layer network.
4. Derive the backpropagation equations for a two-layer MLP.
5. Compare batch, stochastic, and mini-batch gradient descent. When is each preferred?

Module 3: Training Deep Networks

3.1 Activation Functions

Activation functions introduce non-linearity, enabling networks to approximate complex mappings. The choice of activation significantly affects training dynamics.

3.1.1 Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)) \tag{3.1}$$

Output range $(0, 1)$. Smooth and differentiable. Suffers from *vanishing gradients*: for $|z| \gg 0$, $\sigma'(z) \approx 0$, effectively halting learning in deep networks.

3.1.2 Hyperbolic Tangent (tanh)

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \tanh'(z) = 1 - \tanh^2(z) \tag{3.2}$$

Range $(-1, 1)$. Zero-centred, which generally accelerates convergence compared to sigmoid. Still susceptible to vanishing gradients for large $|z|$.

3.1.3 Rectified Linear Unit (ReLU)

$$\text{ReLU}(z) = \max(0, z), \quad \text{ReLU}'(z) = \mathbf{1}[z > 0] \tag{3.3}$$

Computationally efficient; does not saturate for positive inputs, alleviating vanishing gradients. Suffers from the *dying ReLU* problem: neurons with $z < 0$ permanently output 0.

Variants:

- *Leaky ReLU:* $f(z) = \max(\alpha z, z)$, $\alpha \approx 0.01$.
- *Parametric ReLU (PReLU):* α is learned during training.
- *ELU:* $f(z) = z$ for $z > 0$; $\alpha(e^z - 1)$ otherwise. Smooth for negative inputs.

3.1.4 Softmax

$$\text{softmax}(z_j) = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}, \quad j = 1, \ldots, K \tag{3.4}$$

Converts a vector of real values into a probability distribution. Used exclusively in the output layer for multi-class classification.

3.2 Regularisation Techniques

Regularisation prevents overfitting, where the network memorises training data at the expense of generalisation.

3.2.1 L2 Regularisation (Weight Decay)

Penalises large weights by augmenting the loss:

$$\tilde{\mathcal{L}} = \mathcal{L} + \frac{\lambda}{2} \sum_{l} \|\mathbf{W}^{(l)}\|_F^2 \tag{3.5}$$

The gradient update becomes $\mathbf{W}^{(l)} \leftarrow (1 - \eta\lambda)\mathbf{W}^{(l)} - \eta \nabla_{\mathbf{W}^{(l)}} \mathcal{L}$.

3.2.2 Dropout

During each training step, each neuron is independently zeroed out with probability p (typically 0.2–0.5). This forces the network to learn redundant representations and prevents co-adaptation of features. At inference, all neurons are active and weights are scaled by $1 - p$.

3.2.3 Batch Normalisation

Normalises the pre-activations within each mini-batch to have zero mean and unit variance, then applies learnable scale γ and shift β:

$$\hat{z} = \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad \tilde{z} = \gamma \hat{z} + \beta \tag{3.6}$$

Benefits: accelerates training, allows higher learning rates, and has mild regularisation effect.

3.3 Optimisation Algorithms

Standard gradient descent is often insufficient for deep networks. Advanced optimisers adapt the learning rate for each parameter.

3.3.1 Momentum

Accumulates a velocity vector in the direction of persistent gradients:

$$\mathbf{v} \leftarrow \mu \mathbf{v} - \eta \nabla \mathcal{L}, \qquad \mathbf{W} \leftarrow \mathbf{W} + \mathbf{v} \tag{3.7}$$

3.3.2 Adam

Adam (Kingma & Ba, 2015) combines momentum and adaptive learning rates:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t \tag{3.8}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2 \tag{3.9}$$

$$\hat{\mathbf{m}}_t = \mathbf{m}_t / (1 - \beta_1^t), \quad \hat{\mathbf{v}}_t = \mathbf{v}_t / (1 - \beta_2^t) \tag{3.10}$$

$$\mathbf{W}_t = \mathbf{W}_{t-1} - \eta \, \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \epsilon) \tag{3.11}$$

Default hyperparameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

3.4 Hyperparameter Tuning

Key hyperparameters include:

- **Learning rate η:** Most influential. Too large $\Rightarrow$ divergence; too small $\Rightarrow$ slow convergence.
- **Network depth and width:** Affects model capacity and representational power.
- **Batch size:** Larger batches provide stable gradients but may converge to sharp minima.
- **Dropout rate:** Controls regularisation strength.
- **Number of epochs:** Use early stopping: halt training when validation loss stops improving.

3.5 Module 3 — Review Questions

1. Explain the vanishing gradient problem and two strategies to mitigate it.
2. Why is ReLU preferred over sigmoid in hidden layers of deep networks?
3. Describe dropout regularisation. How does it differ at training vs. inference?
4. Derive the Adam update rule and explain the bias correction terms.
5. What is early stopping? How does it prevent overfitting?

Module 4: Advanced Architectures and Applications

4.1 Convolutional Neural Networks (CNNs)

CNNs are specifically designed for data with a grid-like topology, such as images. They exploit three ideas: local receptive fields, weight sharing, and spatial pooling.

4.1.1 Convolutional Layer

A convolution filter (kernel) $\mathbf{K}$ of size $k \times k$ slides across the input $\mathbf{X}$, computing:

$$Y[i,j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X[i+m, j+n] \cdot K[m,n] + b \tag{4.1}$$

Key parameters: kernel size (k), stride (s, step between filter positions), padding (p, border zeros). Output spatial size: $\lfloor (W - k + 2p)/s \rfloor + 1$.

4.1.2 Pooling Layer

Reduces spatial dimensions and introduces translation invariance. Max pooling selects the maximum in each window:

$$P[i,j] = \max_{(m,n) \in \text{window}} X[i+m, j+n] \tag{4.2}$$

4.1.3 Landmark CNN Architectures

LeNet-5 (1998) 5-layer CNN for handwritten digit recognition. 60K parameters.

AlexNet (2012) First deep CNN to win ImageNet. Introduced ReLU and dropout. 60M parameters.

VGGNet (2014) Very deep networks (16–19 layers) using only 3×3 convolutions.

ResNet (2015) Introduced residual (skip) connections: $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$. Enabled training of 150+ layer networks.

EfficientNet (2019) Scales depth, width, and resolution uniformly for state-of-the-art accuracy-efficiency tradeoffs.

4.2 Recurrent Neural Networks (RNNs)

RNNs process sequential data by maintaining a hidden state $\mathbf{h}_t$ that encodes information from previous time steps:

$$\mathbf{h}_t = f(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}), \quad \hat{\mathbf{y}}_t = g(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y) \tag{4.3}$$

4.2.1 Long Short-Term Memory (LSTM)

LSTMs address vanishing gradients through a gating mechanism with a cell state $\mathbf{C}_t$:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f) \qquad \text{(forget gate)} \tag{4.4}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_i) \qquad \text{(input gate)} \tag{4.5}$$

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_C[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_C) \qquad \text{(candidate)} \tag{4.6}$$

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t \qquad \text{(cell state)} \tag{4.7}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_o) \qquad \text{(output gate)} \tag{4.8}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \qquad \text{(hidden state)} \tag{4.9}$$

4.2.2 Gated Recurrent Unit (GRU)

GRU simplifies LSTM with two gates and no separate cell state:

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}; \mathbf{x}_t]) \quad \text{(reset gate)} \tag{4.10}$$

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}; \mathbf{x}_t]) \quad \text{(update gate)} \tag{4.11}$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}[\mathbf{r}_t \odot \mathbf{h}_{t-1}; \mathbf{x}_t]) \tag{4.12}$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \tag{4.13}$$

4.3 Transformer Architecture

Transformers (Vaswani et al., 2017) replaced recurrence with *self-attention*, enabling parallelised training over long sequences:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{4.14}$$

Multi-head attention runs h attention heads in parallel, each attending to different representation subspaces. Transformers are the backbone of BERT, GPT, LLaMA, and other large language models.

4.4 Applications of Neural Networks

Computer Vision Image classification (ResNet), object detection (YOLO, Faster R-CNN), semantic segmentation (U-Net), medical imaging, autonomous driving.

Natural Language Processing Machine translation (Transformer), sentiment analysis, question answering (BERT), text generation (GPT).

Speech Automatic speech recognition (DeepSpeech), text-to-speech synthesis (WaveNet).

Healthcare Tumour detection in radiology, drug discovery, protein structure prediction (AlphaFold).

Robotics Reinforcement learning for locomotion, manipulation, and navigation.

Finance Fraud detection, credit scoring, algorithmic trading, portfolio optimisation.

4.5 Challenges and Future Directions

- **Interpretability:** Black-box nature; explainable AI (XAI) methods such as LIME, SHAP, and Grad-CAM aim to make predictions transparent.
- **Data Efficiency:** Deep networks require large labelled datasets. Few-shot, zero-shot, and self-supervised learning reduce this dependency.
- **Adversarial Robustness:** Networks are vulnerable to imperceptible input perturbations. Adversarial training and certified defences are active research areas.
- **Energy & Environment:** Training frontier models consumes significant compute and energy. Efficient architectures, pruning, and quantisation are important mitigations.
- **Continual Learning:** Overcoming catastrophic forgetting when a model is updated with new tasks while retaining old knowledge.

4.6 Module 4 — Review Questions

1. Explain how the convolutional layer achieves weight sharing and what advantage this provides.
2. Why were residual connections (ResNet) a significant advance? Explain with equations.
3. Compare LSTM and GRU in terms of architecture and computational cost.

4. Describe the self-attention mechanism in the Transformer. What does $\sqrt{d_k}$ scaling achieve?

5. Choose one application domain and explain how a neural network would be designed and trained for it.

Bibliography

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

2. Haykin, S. (2009). *Neural Networks and Learning Machines* (3rd ed.). Pearson.

3. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521, 436–444.

4. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.

5. Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS*, 30.

6. He, K., et al. (2016). Deep residual learning for image recognition. *CVPR*, 770–778.

7. Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.